

DevOps Assignment

Cloud Infrastructure & CI/CD Documentation

Student: *Máté Molnár*

Neptun Cod: *HSMAUW*

GitHub Repository: <https://github.com/momikadragonner/devops-assignment>

Live Application: <https://app-gitea-i138vq.azurewebsites.net>

Application Selection

[Gitea](#) is a self-hosted, open-source Git service that provides the same core functionality as GitHub: code repositories, pull requests, issue tracking, user management, and a full web UI. It is written in Go and distributed as a single binary or Docker image.

Gitea was chosen because:

- It has a clear web frontend (Go + HTML/JS served on port 3000) and a backend database (PostgreSQL).
- It is open-source and publicly available.
- Its Docker image supports complete configuration via environment variables, making it ideal for cloud-native PaaS deployment.

Architecture Analysis

After cloning and analysing the repository, Gitea's architecture is:

Layer	Technology	Role
Web Frontend	Go (HTML/CSS/JS templates, rendered server-side)	User interface: browse repos, manage users, view PRs
Backend API	Go (same binary as the frontend)	REST API, Git operations (HTTP/SSH), authentication
Database	PostgreSQL (primary), also supports MySQL/SQLite	Stores users, repositories metadata, issues, settings
File Storage	Local filesystem (or object storage)	Stores actual Git objects (blobs, trees, commits)

For this deployment, the filesystem storage is provided by the App Service's ephemeral container storage. In production, Azure Blob Storage or a persistent volume would be used.

Cloud Infrastructure (IaC)

Platform: Microsoft Azure

IaC Tool: OpenTofu

[OpenTofu](#) is the open-source fork of Terraform, taught in the lab. All infrastructure is defined in HCL files under terraform/ and provisioned with a single command: `tofu apply`

Provisioned Resources

The following Azure resources are provisioned by terraform/main.tf:

Resource	Azure Type	Purpose
Resource Group rg-gitea-prod-i138vq	azurerm_resource_group	Logical container for all resources
Container Registry acrgiteai138vq	azurerm_container_registry	Stores the custom Gitea Docker images built by CI/CD
PostgreSQL Server psql-gitea-i138vq	azurerm_postgresql_flexible_server	Managed database (DBaaS) stores all Gitea data
PostgreSQL Database gitea	azurerm_postgresql_flexible_server_database	The specific database Gitea connects to
App Service Plan plan-gitea-i138vq	azurerm_service_plan	Linux B2 compute tier for the web app
App Service app-gitea-i138vq	azurerm_linux_web_app	Runs the Gitea Docker container; public-facing URL

Key IaC Design Decisions

Gitea configured entirely via environment variables

Gitea reads the `GITEA__section__key` environment variable pattern to configure every aspect of its behaviour. This avoids baking secrets into the Docker image and allows the same image to run in different environments with different settings.

Sensitive values never committed to Git

The database password is passed via an environment variable (TF_VAR_db_admin_password), not written into any file. OpenTofu marks it as sensitive = true so it is never printed in plan/apply output.

Deployment Platform Justification

Chosen: Azure App Service (PaaS)

Criterion	Rationale
No server management	Azure handles OS patching, host reboots, and health restarts automatically.
Native container support	"Web App for Containers" runs any Docker image from ACR with zero Kubernetes knowledge required.
Deployment simplicity	Updating the running app is a single az webapp config container set call, no cluster, no Helm, no manifests.

Chosen: Azure Database for PostgreSQL — Flexible Server (DBaaS)

Criterion	Rationale
Fully managed	Automated daily backups, point-in-time restore, and automatic minor version upgrades.
SSL enforced	Azure PostgreSQL Flexible Server requires TLS by default; Gitea connects with SSL_MODE=require.
No VM maintenance	No need to patch the database OS, manage storage volumes, or configure replication manually.

CI/CD Pipeline

Tool: GitHub Actions

GitHub Actions was chosen because:

- It is natively integrated with GitHub no external CI service needed.

- It supports reusable Docker actions from the Docker organisation (docker/login-action, docker/build-push-action, etc.).

Pipeline File

.github/workflows/deploy.yml

Trigger: Every push to the main branch (and pull requests for linting only).

Job 1 — Lint & Validate

Step	Tool	What is checked
Docker file lint	hadolint/hadolint-action	Best-practice Dockerfile rules (pinned base versions, etc.)
Format check	tofu fmt -check	Code formatting consistency
Validate	tofu validate	OpenTofu config syntax and provider schema correctness

This job runs on every push and every pull request. If it fails, nothing is built or deployed, giving fast feedback before code is merged.

Job 2 — Build & Push Image

Step	What happens
docker/login-action	Authenticates the runner to ACR using admin credentials
docker/metadata-action	Generates image tags: sha-<7char> (traceability) and latest (convenience)
docker/setup-buildx-action	Enables layer caching via GitHub Actions cache (speeds up subsequent builds)
docker/build-push-action	Builds the Dockerfile from the repository root and pushes both tags to ACR

.

Job 3 — Deploy to App Service

Step	What happens
azure/login	Authenticates with a service principal (AZURE_CREDENTIALS secret)
az webapp config container set	Updates App Service to pull gitea-custom:sha- <short> from ACR
az webapp restart	Restarts the app so the new image is pulled and started
Health check	Polls az webapp show --query state; fails the job if not Running after 30 s

How automatic deployment is guaranteed

- The on: push: branches: [main] trigger means every merge to main fires the pipeline.
- Job 3 only runs if Jobs 1 and 2 both succeed (needs: build).
- There are zero manual steps between a git push and the new version running in Azure.
- A failed pipeline leaves the previous version running, App Service only switches to the new image after a successful pull and start.

Secret Management

No credentials appear in any file in the repository. All sensitive values are stored as GitHub repository secrets and injected at pipeline runtime.